5.2 コエフェクト計算

コエフェクト計算は、線形型システム [8, 9] がベースとなった、コエフェクトと呼ばれる様々な計算的資源の使用状況を分析するためのフレームワークである。線形型システムはデータを線形リソースとして扱う。線形リソースとは、一度使う必要がある一方で、二度と使用することは出来ないデータのことである。例えば、恒等関数 $\lambda x.x$ は変数 x を束縛した後一回だけ使われるため、線形型システムで型付け可能である。しかし、$\lambda x.\lambda y.x$ は束縛された y が使用されず、$\lambda x.x\ x$ は束縛した変数 x が二回使われているため、これらの関数は線形型システムで型付けできない。線形型システムにおいて、線形リソースの対になる概念が非線形リソースである。非線形リソースはモーダル演算子！によってキャプチャされ、使用回数に関する制約が解除されたことを明示できる。ただし、多くのプログラミング言語では殆どの値が非線形リソースであり、その上線形型システムでキャプチャできる計算的リソースの使用状況はあまり詳細なものではなかった。有界線形論理は線形論理の！モダリティを一般化することでこの難点を克服した。有界線形論理では！でキャプチャされたリソースの使用量の上限を定量化することができ、例えば $!_n A$ 型は、最大で n 回使用できる A 型の値をキャプチャする。

2010 年以降、有界線形論理の！モダリティのキャプチャ対象となる計算的リソースの一般化が試みられてきた。Dal Lago と Gaboardi らは、使用回数についてインデックス付けされたモダリティをキャプチャできる計算体系 Linear PCF[34, 35] を提案した。また、De Amorim らは値に与えられるプライバシー情報をキャプチャ可能な計算体系 DFuzz[36] を提案した。また著者らはプログラムのバージョン依存性をキャプチャ可能な計算体系 λ_{VL} [1] を提案した。

モダリティのキャプチャ対象となる計算的リソースの一般化とほぼ同時期に、2013 年から 2014 年頃コエフェクトの概念が考案された。Petricek らはエフェクト計算の圏論的双対としてコエフェクト計算 [37] を定義し、Brunel ら有界線形論理の一般化としてコエフェクト計算 $\ell\mathcal{R}$PCF[19] を定義した。このようにどちら別の背景を持ち独立に生まれた研究であったが、各型システムは基本的に同じ構造を持つ。キャプチャ対象の計算的リソースを代数的構造によって抽象化し、変数に与えられた計算的リソースをデータフローと伴に追跡することにより、そのプログラムがどのような文脈で評価されるべきかを解析する。λ_{VL} の元となった $\ell\mathcal{R}$PCF は有界線形論理における自然数のインデックスを半環に置き換えた。そのためプログラムの型に明示的に計算的リソースのモダリティが与えられている。一方で Petriceik らの型システムにおいては型のモダリティは暗黙的であり、項変数文脈においては保持されているが、型には直接的に現れない。

直近の研究ではコエフェクト計算とエフェクト計算を統一的に表現するフレームワークが研究されてきた。Gaboardi と Katsumata らはエフェクト計算とコエフェクト計算を Graded Modal Type によって統一することを検討した [22]。Orchard らはそれに標準的なプログラミング言語機能 (カインドベースの多相性・パターンマッチ・型強制・代数的データ構造・型制約) を導入した計算体系 `Gr`[20] を提案した。Gr は $\ell\mathcal{R}$PCF と類似のコエフェクト計算 `GrMini`(単純型付きラムダ計算レベルのもの) を計算の核として持ち、言語処理系として Granule を持つ。本研究では `GrMini` から `Gr` への言語拡張を参考にし、λ_{VL} への型・版多相性の導入を行った。

参考文献

[1] Yudai Tanabe, Tomoyuki Aotani, and Hidehiko Masuhara. A context-oriented programming approach to dependency hell. In *Proceedings of the 10th International Workshop on Context-Oriented Programming: Advanced Modularity for Run-time Composition*, COP '18, pp. 8–14, New York, NY, USA, 2018. ACM.

[2] Android Developpers - Platform. https://developer.android.com/about/.

[3] browserify. http://browserify.org/.

[4] npm | building amazing things. https://www.npmjs.com/.

[5] Visualization of npm dependencies. http://npm.anvaka.com/#/view/2d/browserify.

[6] David B. MacQueen. Using dependent types to express modular structure. In *Proceedings of the 13th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL ' 86, p. 277–286, New York, NY, USA, 1986. Association for Computing Machinery.

[7] Barbara H. Liskov and Jeannette M. Wing. A behavioral notion of subtyping. *ACM Trans. Program. Lang. Syst.*, Vol. 16, No. 6, p. 1811–1841, November 1994.

[8] Jean-Yves Girard. Linear logic. *Theor. Comput. Sci.*, Vol. 50, No. 1, p. 1–102, January 1987.

[9] Philip Wadler. Linear types can change the world! In *PROGRAMMING CONCEPTS AND METHODS*. North, 1990.

[10] J. Y. Girard. Une extension de l'interpretation de Godel a l'analyse, et son application a l'elimination des coupures dans l'analyse et la theorie des types. Vol. 63, pp. 63–92, 1971.

[11] John C. Reynolds. Towards a theory of type structure. In *Programming Symposium, Proceedings Colloque Sur La Programmation*, p. 408–423, Berlin, Heidelberg, 1974. Springer-Verlag.

[12] John C. Mitchell and Gordon D. Plotkin. Abstract types have existential type. *ACM Trans. Program. Lang. Syst.*, Vol. 10, No. 3, pp. 470–502, July 1988.

[13] Andreas Rossberg, Claudio V. Russo, and Derek Dreyer. F-ing modules. In *Proceedings of the 5th ACM SIGPLAN Workshop on Types in Language Design and Implementation*, TLDI ' 10, p. 89–102, New York, NY, USA, 2010. Association for Computing Machinery.

[14] David MacQueen. Modules for standard ml. In *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*, LFP '84, pp. 198–207, New York, NY, USA, 1984. ACM.

[15] Robert Harper and Mark Lillibridge. A type-theoretic approach to higher-order modules with sharing. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '94, pp. 123–137, New York, NY, USA, 1994. ACM.

[16] Xavier Leroy. A syntactic theory of type generativity and sharing. *J. Funct. Program.*, Vol. 6, No. 5, pp. 667–698, 1996.

[17] Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, 1st edition, 2002.

[18] G. D. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, Vol. 5, No. 3, pp. 223–255, December 1977.

[19] Aloïs Brunel, Marco Gaboardi, Damiano Mazza, and Steve Zdancewic. A Core Quantitative Coeffect Calculus. In *Programming Languages and Systems*, Lecture Notes in Computer Science, pp. 351–370. Springer, Berlin, Heidelberg, April 2014.

[20] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. Quantitative program reasoning with graded modal types. *Proc. ACM Program. Lang.*, Vol. 3, No. ICFP, July 2019.

[21] Robin Milner, Mads Tofte, and David Macqueen. *The Definition of Standard ML*. MIT Press, Cambridge, MA, USA, 1997.

[22] Marco Gaboardi, Shin-ya Katsumata, Dominic Orchard, Flavien Breuvart, and Tarmo Uustalu. Combining Effects and Coeffects via Grading. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, ICFP 2016, pp. 476–489, New York, NY, USA, 2016. ACM.

[23] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. Quantitative program reasoning with graded modal types. *Proc. ACM Program. Lang.*, Vol. 3, No. ICFP, July 2019.

[24] Apple Developpers - Submit Your Apps to the App Store. https://developer.apple.com/ios/submit/.

[25] Android Developpers - Expanding target API level requirements in 2019. https://android-developers.googleblog.com/2019/02/expanding-target-api-level-requirements.html, February 2019.

[26] Poetry - Python dependency management and packaging made easy. https://python-poetry.org/.

[27] RubyGems.org. https://rubygems.org/.

[28] Bundler: The best way to manage a Ruby application's gems. https://bundler.io/.

[29] Conan, the C / C++ Package Manager for Developers. https://conan.io/.

[30] Introduction: The Cargo Book. https://doc.rust-lang.org/cargo/.

[31] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux J.*, Vol. 2014, No. 239, March 2014.

[32] Rachel Potvin and Josh Levenberg. Why google stores billions of lines of code in a single repository. *Commun. ACM*, Vol. 59, No. 7, p. 78–87, June 2016.

[33] Durham Goode. Facebook Engineering: Scaling Mercurial at Facebook. https://code.fb.com/core-data/scaling-mercurial-at-facebook/, January 2014.

[34] Ugo Dal Lago and Marco Gaboardi. Linear dependent types and relative completeness. In *Proceedings of the 2011 IEEE 26th Annual Symposium on Logic in Computer Science*, LICS ' 11, p. 133–142, USA, 2011. IEEE Computer Society.

[35] Marco Gaboardi, Andreas Haeberlen, Justin Hsu, Arjun Narayan, and Benjamin C. Pierce. Linear dependent types for differential privacy. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ' 13, p. 357–370, New York, NY, USA, 2013. Association for Computing Machinery.

[36] Arthur Azevedo de Amorim, Marco Gaboardi, Emilio Jesús Gallego Arias, and Justin Hsu. Really natural linear indexed type checking. In *Proceedings of the 26nd 2014 International Symposium on Implementation and Application of Functional Languages*, IFL ' 14, New York, NY, USA, 2014. Association for Computing

Machinery.

[37] Tomas Petricek, Dominic Orchard, and Alan Mycroft. Coeffects: A Calculus of Context-dependent Computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, ICFP '14, pp. 123–135, New York, NY, USA, 2014. ACM.

[38] Mark Lillibridge. Translucent sums: A foundation for higher-order module systems. 08 1997.

[39] Andreas Rossberg. 1ml – core and modules united (f-ing first-class modules). *SIGPLAN Not.*, Vol. 50, No. 9, p. 35–47, August 2015.

[40] Andreas Rossberg. *1ML with Special Effects*, Vol. 9600, pp. 336–355. 03 2016.